Building the Lab VM on Proxmox FOR DUMMIES

The same lab, on a Proxmox home-lab host instead of corporate vCenter. Everything after the VM exists is identical — `./setup.sh` doesn't care what hypervisor it's running on.

VM or LXC container?

Use a VM. This one isn't close, and it's worth understanding why, because "just use an LXC, it's lighter" is usually good advice on Proxmox and this is a real exception.

Containerlab isn't just running Docker containers — it's building virtual network topologies. It creates veth pairs, moves interfaces between network namespaces, sets sysctls, manages bridges, and cEOS itself wants to behave like a switch inside its container. That's a pile of privileged kernel-level networking operations.

Running that inside an LXC means nesting: Docker's containers inside a Proxmox container, both sharing the host's kernel. To make it work you progressively turn off the things that make LXC worth using — `nesting=1`, `keyctl=1`, often `fuse=1`, and frequently ending at `lxc.apparmor.profile: unconfined` and dropped capability restrictions. At that point you have something less isolated than a VM and less secure than a normal LXC, and Proxmox's own position is that nested-container setups aren't tested and may break across major version upgrades.

There's also a practical angle: when a lab topology misbehaves, you want to be debugging **OSPF adjacencies**, not whether the problem is a veth that couldn't be created because of a namespace restriction three layers down. A VM removes that entire category of confusion.

The overhead you're paying for that is roughly 5–15% CPU and a few hundred MB of RAM for the guest kernel. On a home lab host that's a fine trade for "it just works."

LXC is still the right call for most of your other home-lab services — this is specifically about workloads that manipulate kernel networking.

VM settings

Proxmox web UI → **Create VM**.

Tab	Setting	Value
General	Name	`clab-lab` (or whatever)
OS	ISO image	Debian 13 (Trixie) — see note below
System	Machine	`q35`
System	BIOS	OVMF (UEFI) or SeaBIOS, either is fine
System	Qemu Agent	✔ Enable (then `apt install qemu-guest-agent` in the guest)
Disks	Bus/Device	**VirtIO Block** (or SCSI w/ VirtIO SCSI single)
Disks	Size	64 GB, and check **Discard** if on SSD/ZFS
CPU	Cores	8 (or as many as you can spare)
CPU	Type	`host` — see note
Memory	RAM	16–32 GB (see sizing below)
Network	Model	**VirtIO (paravirtualized)**
Network	Bridge	`vmbr0` (your LAN bridge, so it gets a DHCP lease you can SSH to)

CPU type `host` matters — it passes your real CPU flags through instead of emulating a generic model. Not strictly required for cEOS (it's a container, no nested virtualization involved), but if you later want **vrnetlab** to run true VM-based images (Cisco IOS-XE, Juniper vMX, Palo Alto), those need nested virt, which needs `host` plus nesting enabled on the Proxmox host:

```
# on the Proxmox host, check nested virt is on:
cat /sys/module/kvm_intel/parameters/nested   # or kvm_amd
```

RAM sizing: each cEOS node wants 0.5–1 GB. 16 GB comfortably runs the 2-node test lab plus a 6-router ring; go to 32 GB if you want a full spine-leaf with room to spare. Leave **ballooning off** — you don't want memory pulled out from under running network nodes.

Getting a Debian 13 ISO

Proxmox has no built-in Debian image, so upload one:

1. Download the **DVD-1** ISO from https://www.debian.org/distrib/ (not netinst — netinst needs working internet mid-install and leaves you with a bare system if the network isn't up yet).

2. Proxmox UI → your node → **local (storage)** → **ISO Images** → **Upload**.

Or pull it directly on the Proxmox host, which is usually faster:

```
cd /var/lib/vz/template/iso
wget https://cdimage.debian.org/debian-cd/current/amd64/iso-dvd/debian-13.6.0-amd64-DVD-1.iso
```

(check the current filename at that URL first — point releases change)

Installing Debian

Standard installer. The only choices that matter:

- **Software selection:** uncheck all desktop environments; keep **SSH server** and **standard system utilities**
- **Partitioning:** guided, use entire disk — it's a lab VM

After first boot:

```
sudo apt install -y qemu-guest-agent
sudo systemctl enable --now qemu-guest-agent
```

That's what makes the Proxmox UI show the VM's IP address and lets it do clean shutdowns.

Then what?

Find the VM's IP (Proxmox **Summary** tab shows it once the guest agent is running, or `ip a` in the console), SSH in, and follow the main `README.md` Quick Start — get the files onto the VM per `distributing.md`, then:

```
cd ~/clab-bootstrap
./setup.sh
```

`setup.sh` installs everything else: `gum` (offered, optional), `uv`, Docker CE, Containerlab, the Python automation tooling, and it can deploy the test topology and walk you through the automation demo.

Differences from the VMware build

Barely any, and none you have to act on:

- **Interface name** is `ens18` (VirtIO) instead of `ens192` (VMXNET3). The scripts detect this rather than hardcoding it, and DHCP from your home router means there's no manual network config to do at all — the whole `/etc/network/interfaces` dance in the original build notes was a workaround for a corporate network that isn't your problem here.
- **No jump host, no DNAT, no VPN.** The VM is on your LAN, you SSH straight to it. The "Access Architecture" contortions in the original MOP were entirely about getting through corporate network segmentation.
- **You control the hypervisor**, so snapshots are free — take one after `setup.sh` finishes and before you start breaking things in labs. That's genuinely the biggest quality-of-life win over the work VM.